UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA
DEPARTAMENTO DE CIÊNCIAS EXATAS
ÁREA DE INFORMÁTICA
MÓDULO: EXA 805 – ALGORITMOS E PROGRAMAÇÃO II
PROFESSOR: ROBERTO ALMEIDA BITTENCOURT

LISTA DE EXERCÍCIOS – SEGUNDA UNIDADE

1. Explique, ***com suas palavras***, o que é polimorfismo, o que se objetiva com a sua utilização, como ele é materializado em Java, além de dar dois exemplos de polimorfismo diferentes dos utilizados nos slides das aulas (apenas a descrição das classes e métodos; não é necessário código-fonte).

2. Explique, ***com suas palavras***, as similaridades e as diferenças entre classes abstratas e interfaces. Dê exemplos de situações reais, utilizando diagramas de classes na sua ilustração, onde você usaria cada um destes tipos de dados. Os exemplos devem ser diferentes dos slides usados em sala de aula.

3. Explique, ***com suas palavras***, as diferenças entre herança de especificação, herança de extensão e herança polimórfica. Ilustre suas explicações com exemplos de problemas reais (diferentes dos usados nos slides em sala de aula), utilizando diagramas de classes na sua ilustração.

4. Explique, ***com suas palavras***, as diferenças entre técnicas de reuso de software baseadas em herança e as baseadas em associação (inclusive agregação e composição), explicitando as vantagens e desvantagens de cada uma das duas técnicas. Dê exemplos de duas situações reais em que é melhor usar associação e outras duas situações reais em que é melhor usar herança para promover o reuso de código-fonte.

5. Explique, ***com suas palavras***, o padrão de design DAO. Sua explicação deve apontar os problemas que o padrão tenta resolver, a solução geral (explique, com clareza e concisão, os papéis desempenhados pelas classes e interfaces do padrão), além de seus prós e contras.

Nos exercícios de rastreamento a seguir, analise o código cuidadosamente e determine a saída ou o comportamento esperado, justificando suas respostas quando solicitado.

Nos exercícios de implementação a seguir, escreva o código Java necessário para atender aos requisitos apresentados. Quando solicitado, inclua também um exemplo de utilização.

6. Considere o código:

```
abstract class Midia {
    public abstract int duracao();
}

class Filme extends Midia {
    private int minutos;

    public Filme(int minutos) {
        this.minutos = minutos;
    }

    @Override
    public int duracao() {
        return minutos;
    }
}

class Podcast extends Midia {
    private int episodios;
    private int minutosPorEpisodio;

    public Podcast(int episodios, int minutosPorEpisodio) {
        this.episodios = episodios;
        this.minutosPorEpisodio = minutosPorEpisodio;
    }

    @Override
    public int duracao() {
        return episodios * minutosPorEpisodio;
    }
}
```

Considere o trecho:

```
Midia[] itens = {
    new Filme(120),
    new Podcast(8, 35),
    new Filme(95)
};

int total = 0;

for (Midia item : itens) {
    System.out.println(item.duracao());
    total += item.duracao();
}

System.out.println(total);
```

a) Qual é a saída completa?

b) Por que o vetor pode armazenar objetos de classes diferentes?

c) O que determina qual implementação de `duracao()` será executada em cada iteração?

7. Considere:

```
interface Exportavel {
    String exportar();
}

abstract class Relatorio implements Exportavel {
    protected String titulo;

    public Relatorio(String titulo) {
        this.titulo = titulo;
    }

    public String getTitulo() {
        return titulo;
    }
}

class RelatorioTexto extends Relatorio {
    public RelatorioTexto(String titulo) {
        super(titulo);
    }

    @Override
    public String exportar() {
        return "TXT: " + titulo;
    }
}

class RelatorioHtml extends Relatorio {
    public RelatorioHtml(String titulo) {
        super(titulo);
    }

    @Override
    public String exportar() {
        return "<h1>" + titulo + "</h1>";
    }
}
```

Analise:

```
Exportavel a = new RelatorioTexto("Vendas");
Exportavel b = new RelatorioHtml("Estoque");

System.out.println(a.exportar());
System.out.println(b.exportar());
```

a) Qual é a saída?

b) Por que `Exportavel` pode ser usado como tipo das variáveis `a` e `b`?

c) Qual seria o problema de tentar instanciar diretamente `Relatorio`?

8. Implemente uma interface `Notificavel` com um método `enviar(String mensagem)`.

Implemente duas classes que realizem essa interface:

- `NotificacaoEmail`, que recebe um endereço de e-mail no construtor.
- `NotificacaoSMS`, que recebe um número de telefone no construtor.

Cada classe deve implementar `enviar()`, imprimindo a mensagem e o respectivo destinatário.

Depois, escreva um método `notificarTodos()` que receba um vetor de `Notificavel` e uma mensagem, enviando a mensagem para todos os elementos do vetor.

O objetivo é permitir adicionar novas formas de notificação sem precisar modificar o método `notificarTodos()`.

9. Implemente uma classe abstrata `Colaborador`, com atributos privados para nome e salário-base, um construtor e métodos de acesso.

A classe deve declarar um método abstrato `calcularBonus()`.

Implemente duas subclasses:

- `ColaboradorTecnico`: bônus de 8% do salário-base.
- `ColaboradorGerente`: bônus de 15% do salário-base.

Crie também um método que receba um vetor de `Colaborador` e calcule a soma dos bônus de todos os elementos, sem verificar explicitamente o tipo de cada objeto com `instanceof`.

10. Implemente uma interface `ComparavelPorPreco` com um método que permita comparar o preço de um objeto com o de outro objeto do mesmo tipo.

Implemente uma classe `Produto`, com nome e preço, que realize essa interface.

Escreva um método que receba dois produtos e retorne o produto de menor preço. Se os preços forem iguais, o método poderá retornar qualquer um deles.

Explique brevemente por que definir um contrato por meio de uma interface pode facilitar a extensão do programa.

11. Considere:

```
public class TesteExcecoes {
    public static void executar(int n) {
        try {
            System.out.println("Início");

            if (n < 0) {
                throw new IllegalArgumentException();
            }

            if (n == 0) {
                throw new ArithmeticException();
            }

            System.out.println(100 / n);
        } catch (ArithmeticException e) {
            System.out.println("Erro aritmético");
        } catch (IllegalArgumentException e) {
            System.out.println("Argumento inválido");
        } finally {
            System.out.println("Fim");
        }
    }
}
```

Determine a saída de cada chamada, considerada separadamente:

a) `executar(10)`

b) `executar(0)`

c) `executar(-2)`

Explique também a finalidade do bloco `finally`.

12. Considere:

```
public class Operacoes {
    public static void metodoA() {
        System.out.println("A1");
        metodoB();
        System.out.println("A2");
    }

    public static void metodoB() {
        System.out.println("B1");
        throw new IllegalStateException();
    }

    public static void main(String[] args) {
        try {
            metodoA();
        } catch (IllegalStateException e) {
            System.out.println("Exceção capturada");
        }

        System.out.println("Fim do programa");
    }
}
```

Responda:

a) Qual é a saída completa?

b) A instrução que imprime `"A2"` é executada? Explique.

c) Em qual método a exceção é lançada e em qual método ela é capturada?

13. Implemente uma classe `Cadastro` com o método estático `validarIdade(int idade)`.

O método deve lançar `IllegalArgumentException` quando a idade for negativa. Caso contrário, deve imprimir `"Idade válida"`.

Escreva também um método `main` que teste uma idade válida e uma idade inválida, capturando a exceção e imprimindo `"Idade inválida"` quando ela ocorrer.

A exceção deve ser tratada no `main`, e não dentro de `validarIdade()`.

14. Implemente um método estático `obterElemento(int[] valores, int indice)` que retorne o elemento na posição indicada.

Se o índice estiver fora dos limites do vetor, o método deve lançar uma exceção apropriada.

Escreva um programa que chame o método com um índice válido e outro inválido, tratando a exceção no código que faz as chamadas. Não é necessário tratar o caso em que o próprio vetor é `null`.

15. Implemente uma classe `Conta` com um atributo privado `saldo`, inicializado pelo construtor.

O método `sacar(double valor)` deve:

- Lançar `IllegalArgumentException` se o valor for menor ou igual a zero.
- Lançar `IllegalStateException` se o valor for superior ao saldo.
- Subtrair o valor do saldo quando o saque for válido.

Escreva um exemplo de utilização que tente realizar um saque válido e um saque superior ao saldo, tratando as exceções com mensagens diferentes.

16. Considere:

```
class Caixa<T> {
    private T conteudo;

    public Caixa(T conteudo) {
        this.conteudo = conteudo;
    }

    public T getConteudo() {
        return conteudo;
    }

    public void substituir(T novoConteudo) {
        conteudo = novoConteudo;
    }
}
```

Analise:

```
Caixa<String> c1 = new Caixa<>("Java");
Caixa<Integer> c2 = new Caixa<>(42);

c1.substituir("Python");

System.out.println(c1.getConteudo());
System.out.println(c2.getConteudo());
System.out.println(c1.getConteudo().length());
```

Responda:

a) Qual é a saída?

b) Qual é o tipo do valor retornado por `c1.getConteudo()` e por `c2.getConteudo()`?

c) Seria possível executar `c1.substituir(100)`? Justifique.

17. Considere:

```
public class Util {
    public static <T> void exibirDois(T primeiro, T segundo) {
        System.out.println(primeiro);
        System.out.println(segundo);
    }

    public static <T> T escolherPrimeiro(T a, T b) {
        return a;
    }

    public static void main(String[] args) {
        exibirDois("Ana", "Bruno");
        exibirDois(10, 20);

        String resultado = escolherPrimeiro("A", "B");
        System.out.println(resultado);
    }
}
```

Responda:

a) Qual é a saída completa?

b) Qual é o papel do parâmetro de tipo `T` nos dois métodos?

c) Por que o método `escolherPrimeiro()` pode retornar um valor sem conhecer antecipadamente o tipo concreto dos argumentos?

18. Implemente uma classe genérica `Par<A, B>` que armazene dois valores, possivelmente de tipos diferentes.

A classe deve ter:

- Um construtor que receba os dois valores.
- Um método `getPrimeiro()` que retorne o primeiro valor.
- Um método `getSegundo()` que retorne o segundo valor.

Escreva um exemplo que represente um produto por meio de seu nome (`String`) e seu código numérico (`Integer`).

19. Implemente um método estático genérico:

```
public static <T> int localizar(T[] elementos, T procurado)
```

O método deve retornar o índice da primeira ocorrência de `procurado` no vetor, ou `-1` se o elemento não estiver presente. Utilize `equals()` para comparar os elementos e considere que o vetor não contém valores `null`.

Teste o método com um vetor de `String` e outro de `Integer`.

20. Implemente uma classe genérica `Estatisticas<T extends Number>` que receba um vetor de valores numéricos.

A classe deve oferecer:

- Um método `media()` que retorne a média dos valores como `double`.
- Um método `maiorQue(double limite)` que retorne quantos elementos são estritamente maiores que o limite.

Considere vetores não vazios e sem elementos `null`.

Explique por que a declaração `T extends Number` é útil para implementar essas operações.

21. Considere:

```
import java.util.ArrayList;
import java.util.List;

public class TesteLista {
    public static void main(String[] args) {
        List<String> nomes = new ArrayList<>();

        nomes.add("Ana");
        nomes.add("Bruno");
        nomes.add(1, "Carla");
        nomes.set(0, "Daniel");

        System.out.println(nomes.get(1));
        System.out.println(nomes.size());
        System.out.println(nomes.contains("Ana"));
        System.out.println(nomes);
    }
}
```

Responda:

a) Qual é a saída completa?

b) Qual é a diferença entre `add()` e `set()` nesse contexto?

c) Qual é a diferença entre `List` e `ArrayList` no código?

22. Considere:

```
import java.util.HashMap;
import java.util.Map;

public class TesteMapa {
    public static void main(String[] args) {
        Map<String, Integer> notas = new HashMap<>();

        notas.put("Ana", 8);
        notas.put("Bruno", 6);
        notas.put("Ana", 9);

        System.out.println(notas.get("Ana"));
        System.out.println(notas.get("Bruno"));
        System.out.println(notas.containsKey("Carla"));
        System.out.println(notas.size());
    }
}
```

Responda:

a) Qual é a saída completa?

b) Por que a segunda inserção de `"Ana"` não aumenta o tamanho do mapa?

c) Qual é a diferença entre consultar um valor por `get()` e verificar uma chave por `containsKey()`?

23. Implemente uma classe `FilaAtendimento` que utilize uma estrutura apropriada do Java Collections Framework para armazenar nomes de clientes.

A classe deve oferecer:

- `entrar(String nome)`: adiciona um cliente ao final da fila.
- `atender()`: remove e retorna o cliente que está há mais tempo na fila.
- `proximo()`: consulta o próximo cliente sem removê-lo.
- `estaVazia()`: informa se há clientes aguardando.

Caso `atender()` ou `proximo()` sejam chamados quando a fila estiver vazia, o programa deve tratar essa situação de forma apropriada.

Escreva um exemplo que insira três clientes e atenda dois deles.

24. Implemente uma classe `CatalogoProdutos` que utilize um `Map<String, Double>` para relacionar códigos de produtos a preços.

A classe deve oferecer:

- `cadastrar(String codigo, double preco)`: registra ou atualiza o preço de um produto.
- `consultarPreco(String codigo)`: retorna o preço cadastrado ou indica que o código não existe.
- `remover(String codigo)`: remove um produto do catálogo.
- `quantidade()`: retorna a quantidade de códigos cadastrados.

Considere que os preços são não negativos. Escreva um exemplo que cadastre três produtos, atualize um preço, consulte um código e remova outro produto.

25. Implemente um método estático:

```
public static List<Integer> selecionarPares(Collection<Integer> numeros)
```

O método deve retornar uma nova lista contendo somente os números pares da coleção recebida, sem modificar a coleção original.

A lista retornada deve preservar a ordem em que os números pares aparecem na coleção de entrada.

Teste o método utilizando uma `ArrayList<Integer>` e uma coleção do tipo `LinkedList<Integer>`.

Explique por que declarar o parâmetro como `Collection<Integer>` permite utilizar o método com diferentes implementações de coleções.

26. Considere um arquivo `notas.txt` contendo uma nota por linha, representada como texto. Escreva um método `calcularMedia()` que leia todas as notas, calcule e retorne a média aritmética. Utilize `BufferedReader` e considere que o arquivo contém pelo menos uma nota válida por linha.
27. Implemente um método `copiarArquivo(String origem, String destino)` que copie o conteúdo de um arquivo para outro utilizando `FileInputStream` e `FileOutputStream`. O método deve funcionar também para arquivos binários, como imagens e PDFs. Utilize um vetor de bytes para realizar a cópia e declare as exceções necessárias.
28. Considere uma classe `Produto`, com os atributos `codigo` (inteiro) e `preco` (double). Escreva um método que grave três objetos `Produto` em um arquivo chamado `produtos.dat`, utilizando `ObjectOutputStream`. Em seguida, escreva outro método que leia os três objetos do arquivo e exiba seus dados na tela, utilizando `ObjectInputStream`.
29. Considere uma classe `Funcionario`, com os atributos `nome`, `salario` e `senha`. Implemente a classe para permitir a serialização dos objetos, de modo que o atributo `senha` não seja persistido no arquivo. Escreva também um trecho de código que grave um objeto e depois o recupere, explicando o que acontece com o atributo `senha` após a leitura.
30. Considere uma classe `Configuracao`, com os atributos `idioma` (String) e `volume` (inteiro). Implemente a interface `Externalizable` para que esses atributos sejam gravados e recuperados manualmente. Inclua o construtor público sem argumentos e os métodos `writeExternal()` e `readExternal()`. Explique por que a ordem de leitura dos atributos deve corresponder à ordem de gravação.